Clasificación de sentimientos en reseñas de películas de IMDb con una red neuronal recurrente LSTM

Juan Sebastian Hernandez, Juan Carlos Echeverri, Juan Fernando Contreras

1. Introducción

Las reseñas de películas en IMDb son texto libre escrito por usuarios, y su polaridad (positiva o negativa) sirve para resumir la opinión del público o para alimentar sistemas de recomendación. Con decenas de miles de reseñas, etiquetarlas a mano no es viable. La tarea es difícil porque el sentimiento depende del orden y del contexto: una negación, una ironía o una queja al final pueden cambiar el sentido de un texto lleno de elogios.

Los primeros clasificadores de sentimiento representaban cada reseña como una bolsa de palabras. Con esa representación y una SVM, Pang *et al.* [1] clasificaron 1400 reseñas de cine con 82,9 % de exactitud, aunque así se pierde el orden de las palabras. Maas *et al.* [2] publicaron un corpus de 50 000 reseñas de IMDb y, al combinar vectores de palabras aprendidos con la bolsa de palabras, llegaron a 88,89 %. Las redes recurrentes (RNN) leen la reseña token a token y resumen lo leído en un estado oculto. En una RNN simple el gradiente se desvanece en secuencias largas [3]; la LSTM [4] lo mitiga con compuertas que deciden qué información se conserva y cuál se olvida en cada paso.

El objetivo de este trabajo es clasificar reseñas de IMDb como positivas o negativas con una red recurrente. Para ello construimos un pipeline que limpia y tokeniza el texto, arma el vocabulario solo con datos de entrenamiento y convierte cada reseña en una secuencia de índices. Con esas secuencias entrenamos una LSTM con una capa de embedding aprendida desde cero y la evaluamos en un conjunto de prueba aislado con *precision*, *recall* y F1 por clase.

2. Metodología

Usamos el archivo movie.csv del conjunto *IMDB Movie Ratings Sentiment Analysis* de Kaggle [5]: 40 000 reseñas en inglés con el texto y una etiqueta binaria, 1 para las positivas (19 981) y 0 para las negativas (20 019). Encontramos 277 textos repetidos de forma exacta, ninguno con etiquetas distintas, y dejamos una sola copia de cada uno (39 723 reseñas) para que la misma reseña no quede en entrenamiento y en prueba. La división es estratificada, con semilla 42: 70 % para entrenamiento (27 806), 15 % para validación (5958) y 15 % para prueba (5959).

Para tokenizar decodificamos las entidades HTML, cambiamos etiquetas como
 por espacios y pasamos el texto a minúsculas. Una expresión regular conserva palabras, números, contracciones como *didn't* y los signos ! y ?, que suelen acompañar opiniones fuertes. El vocabulario tiene 20 000 entradas: los 19 998 tokens más frecuentes en entrenamiento, <PAD> para el relleno y <UNK> para las palabras fuera del vocabulario. No truncamos las reseñas. El DataLoader rellena cada lote hasta la longitud de su reseña más larga y entrega también la longitud real de cada una.

El modelo tiene tres capas. La primera es un embedding que asigna a cada índice un vector entrenable de 64 dimensiones. El enunciado menciona Word2Vec o GloVe [6]; preferimos aprender los vectores junto con el clasificador para que se ajusten a la tarea de sentimiento y no depender de un vocabulario externo. La segunda es una LSTM unidireccional de una capa con estado oculto de 64 unidades, elegida porque retiene contexto en reseñas de cientos de tokens [4]. Tomamos su salida en el último token real de cada reseña, de modo que el relleno no influye en la predicción. La tercera es una capa densa de 64 a 1 que produce un logit, y su sigmoide es la probabilidad de la clase positiva. El modelo suma 1 313 345 parámetros, el 97,5 % de ellos en el embedding.

Entrenamos con Adam [7], tasa de aprendizaje de 10^{-3} y lotes de 32 reseñas. La pérdida es BCEWithLogitsLoss de PyTorch, que integra la sigmoide en la entropía cruzada binaria y es numéricamente más estable que aplicarlas por separado. Permitimos hasta 5 épocas con parada temprana: si la pérdida de validación no mejora en 2 épocas seguidas, el entrenamiento se detiene, y al final se restauran los pesos de la época con menor pérdida de validación. El conjunto de prueba solo se usa en la evaluación final. Todo corrió en CPU con 4 hilos.

En prueba, una reseña se clasifica como positiva si su probabilidad es al menos 0,5. Reportamos *precision*, *recall* y F1 por clase, su promedio macro y la exactitud. Las clases están casi balanceadas, así que la exactitud es informativa; *precision* y *recall* por clase muestran además hacia qué lado se equivoca el modelo, y el F1 macro da el mismo peso a las dos polaridades.

3. Resultados

El entrenamiento tardó 8,76 minutos y completó las 5 épocas sin que se activara la parada temprana. En la Fig. 1a, la pérdida de entrenamiento baja en cada época, de 0,586 a 0,281. La de validación baja de 0,577 a 0,344, con un repunte leve en la época 3 (de 0,441 a 0,448). La mejor época fue la 5, la última permitida, aunque entre la 4 y la 5

la pérdida de validación bajó apenas 0,010.

La Tabla 1 resume el desempeño en las 5959 reseñas de prueba: exactitud de 86,6 % y F1 macro de 0,866. La pérdida de prueba (0,337) es muy cercana a la de validación (0,344). Las dos clases tienen un F1 parecido (0,861 y 0,871), pero no el mismo balance entre *precision* y *recall*: el modelo recupera el 90,2 % de las reseñas positivas y el 83,1 % de las negativas. En la matriz de confusión de la Fig. 1b, 502 reseñas negativas quedaron como positivas, frente a 294 errores en el sentido contrario. En total predijo 3194 positivas (53,6 %) cuando había 2986 (50,1 %).

Tabla 1. Métricas por clase en el conjunto de prueba con los pesos de la época 5. Exactitud: 0,8664; pérdida BCE: 0,3369.

Clase	*Precision*	*Recall*	F1	Reseñas
0 (negativa)	0,8937	0,8311	0,8613	2973
1 (positiva)	0,8428	0,9015	0,8712	2986
Promedio macro	0,8683	0,8663	0,8662	5959

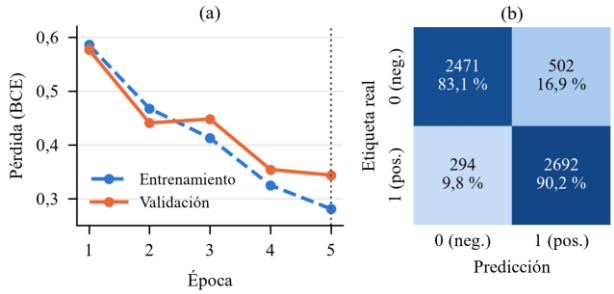


Fig. 1. (a) Pérdida de entropía cruzada binaria por época en entrenamiento (línea discontinua) y validación (línea continua); la línea punteada marca la época 5, cuyos pesos se restauraron. (b) Matriz de confusión en prueba (5959 reseñas); cada celda muestra el número de reseñas y el porcentaje respecto a la clase real.

La Tabla 2 muestra la reseña de prueba más corta de cada combinación de etiqueta real y predicción. Los aciertos contienen palabras de opinión explícitas (*unwatchable*, *great job*). En los errores el sentimiento es indirecto: "Read the book, forget the movie!" es negativa sin usar palabras negativas, y la reseña positiva mal clasificada elogia la película y cierra quejándose de que no está disponible en video en Australia ("What a waste").

Tabla 2. Reseña de prueba más corta de cada combinación de etiqueta real y predicción, con la probabilidad asignada a la clase positiva. Las dos últimas filas son errores; [...] indica texto omitido.

Reseña	Real	Pred.	P(pos.)
Unwatchable. You can't even make it past the first three minutes. And this is coming from a huge Adam Sandler fan!!1	0	0	0,320
As usual, Sean Connery does a great job. Lawrence Fishburn is good, but I have a hard time not seeing him as Ike Turner.	1	1	0,960
Read the book, forget the movie!	0	1	0,737
What an amazing film. With very little dialogue, the whole story is told with glances and body language. [...] What a waste.	1	0	0,174

4. Discusión

El modelo no llegó a converger: la pérdida de validación siguió bajando hasta la época 5, que era el límite fijado. La diferencia entre la pérdida de validación y la de entrenamiento pasó de −0,009 en la época 1 a 0,063 en la época 5, el patrón que acompaña a un aumento de la varianza [8]. Buena parte de la capacidad está en el embedding (97,5 % de los parámetros), entrenado desde cero con 27 806 reseñas, y las palabras poco frecuentes reciben vectores ajustados a las pocas reseñas en que aparecen. El repunte de la época 3 es una sola observación y no alcanza para hablar de sobreajuste. Como la pérdida de prueba quedó al nivel de la de validación, elegir la época con validación no parece haber inflado el desempeño.

Los errores de la Tabla 2 podrían deberse a que el modelo resume la reseña con el estado oculto del último token, que en una LSTM unidireccional está más influido por lo leído al final. La reseña positiva que cierra con "What a waste" recibe una probabilidad positiva de 0,174. En el acierto negativo, el elogio final a Adam Sandler deja esa probabilidad en 0,320: una seguridad de 0,68 en la clase correcta, frente a 0,96 en el acierto positivo. Son dos ejemplos y no prueban la hipótesis; habría que medir cuánto cambia la predicción al quitar la última frase. Las reseñas cortas sin vocabulario de opinión, como "Read the book, forget the movie!", exigen conocimiento que un modelo entrenado desde cero con estos datos difícilmente adquiere.

Nuestro 86,6 % queda por debajo del 88,89 % de Maas *et al.* [2], pero la comparación es solo orientativa. Ellos separan entrenamiento y prueba por película; nuestra división es por reseña, así que reseñas de una misma película pueden quedar en ambos conjuntos y hacer que el resultado sea algo optimista. Además, reportamos una sola corrida con una sola semilla, sin desviación estándar ni otra configuración para comparar, y no sabemos qué parte de la diferencia entre clases o del repunte de la época 3 es ruido. El umbral de 0,5 no se ajustó y la deduplicación solo detecta textos idénticos.

Proponemos entrenar más épocas y dejar que la parada temprana decida, añadir *dropout* sobre el embedding e inicializarlo con GloVe [6] para que las palabras raras partan de vectores con significado. También conviene comparar con una GRU [9] y con una LSTM bidireccional, que lee la reseña también de atrás hacia adelante. Repetir cada configuración con varias semillas permitiría reportar promedio y desviación estándar, y ajustar el umbral en validación podría corregir el sesgo hacia la clase positiva.

Referencias

[1] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2002, pp. 79–86.

[2] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proc. 49th Annu. Meeting Assoc. Comput. Linguistics: Human Lang. Technol. (ACL-HLT)*, 2011, pp. 142–150.
[3] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, 1994.
[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
[5] M. Yasser H., "IMDB movie ratings sentiment analysis," Kaggle, 2022. [Online]. Available: https://www.kaggle.com/datasets/yasserh/imdb-movie-ratings-sentiment-analysis
[6] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1532–1543.
[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
[8] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
[9] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1724–1734.